1️⃣ From hard-coded loops to “combining values”

In `programReduce01.js`, the code uses two separate loops:

- One to **add up all the numbers**
- One to **find the largest number**

Each loop repeatedly updates a running value:

```js
total = total + givenArray[i];
```

and

```js
if (givenArray[i] > max) {
    max = givenArray[i];
}
```

programReduce01

Here, the loop is doing **two jobs at once**:

1. Walking through the array
2. Deciding how to combine values

This works, but it is not flexible.

2️⃣ Making the “combine step” visible

In `programReduce02.js`, the key idea becomes clearer:

```js
const newTotal = total + givenArray[i];
total = newTotal;
```

and

```js
const newMax = (givenArray[i] > max) ? givenArray[i] : max;
max = newMax;
```

programReduce02

Now you can see that:

Each loop step **takes the previous result** and **combines it with the next value** to produce a new result.

This is the heart of what `reduce` does.

3 Turning the combining logic into functions

In `programReduce03.js`, the combination logic is extracted:

In `programReduce03.js`, the combination logic is extracted:

```js
function add(currentSum, currentValue) {
    return currentSum + currentValue;
}

function keepLarger(currentHighest, currentValue) {
    return (currentValue > currentHighest) ? currentValue : currentHighest;
}
```

These are then used inside the loops:

```js
total = combineFunction(total, givenArray[i]);
max = combineFunction(max, givenArray[i]);
```

programReduce03

Now the loop no longer knows how to add or compare —
it simply calls a **callback function** and trusts the answer.

4 One generic reduce function

In `programReduce04.js`, this idea becomes a true `reduce`:

```js
function myReduce(givenArray, combineFunction, startingValue) {
    let accumulator = startingValue;

    for (let i = 0; i < givenArray.length; i++) {
        accumulator = combineFunction(accumulator, givenArray[i]);
    }
    return accumulator;
}
```

programReduce04

Now the algorithm is:

“Take a running value (accumulator) and keep combining it with each array element using a callback.”

The callback decides **how** the combination happens.

5 Callback functions on the fly

In `programReduce05.js`, the callback is written inline:

```js
const sum = myReduce(numberArr, (currentSum, currentValue) => currentSum + current

const max = myReduce(numberArr,
    (currentHighest, currentValue) => (currentValue > currentHighest ? currentValu
    numberArr[0]);
```

Refer to actual code for the complete code

This shows that:

The reduce algorithm does not care what the rule is — it just calls the callback again and again.

6 JavaScript's built-in `Array.reduce()`

Finally, `programReduce06.js` uses the real thing:

```js
const sum = numberArr.reduce((currentSum, currentValue) => currentSum + currentValue

const max = numberArr.reduce(
    (currentHighest, currentValue) => (currentValue > currentHighest ? currentValue
    numberArr[0]
);
```

Refer to actual code for the complete code

This is exactly the same as `myReduce`, except JavaScript provides it for you.

🎯 Final Concept Students Should Understand

`Array.reduce()` separates **two responsibilities**:

Part	Responsibility
`reduce`	Loops through the array and keeps track of the accumulator
Callback	Decides how to combine the accumulator with each value

So when you write:

```js
numbers.reduce((a, b) => a + b, 0)
```

you are not telling JavaScript **how to loop** —
you are only telling it **how to combine two values**.

That is the core idea of `reduce`.

~ END ~